

개발기관	2025-XXXX-XX
배정연번	

개발이력	
최초 개발	2025-11-01
1차 업데이트	1.
2차 업데이트	2.
3차 업데이트	3.

「 AI 제조 기술 교육-25-XX 」

ChatGPT TDD 개발 - Cursor AI 단계별 프롬프트

제 출 일 :

소 속 :

성 명 :

※ 워크북 제출시 반드시 파일명을 ○○○○(제출년월일8자리)_○○○(성명).hwp로 하여

주십시오.(예시 : 20250128_홍길동.hwp)

개발일자 : 2025-11-01

ChatGPT TDD 개발 - Cursor AI 단계별 프롬프트

📖 목차

1. [프로젝트 초기 설정](#)
 2. [Phase 1: STT 기능 \(TDD\)](#)
 3. [Phase 2: GPT 연동 \(TDD\)](#)
 4. [Phase 3: TTS 기능 \(TDD\)](#)
 5. [Phase 4: Streamlit UI 구현](#)
 6. [Phase 5: 통합 및 리팩토링](#)
-

Phase 0: 프로젝트 초기 설정

프롬프트 0-1: 프로젝트 구조 생성

- ChatGPT이라는 음성 비서 챗봇 프로젝트를 시작합니다.
- 다음 구조로 프로젝트를 생성해주세요:
-
- ChatGPT/
 - |—— src/
 - | |—— __init__.py
 - | |—— stt/
 - | | |—— __init__.py
 - | | |—— whisper_service.py
 - | |—— llm/
 - | | |—— __init__.py
 - | | |—— gpt_service.py
 - | |—— tts/
 - | | |—— __init__.py

- | | └── gtts_service.py
- | └── utils/
- | └── __init__.py
- | └── audio_utils.py
- └── tests/
- | └── __init__.py
- | └── test_stt.py
- | └── test_llm.py
- | └── test_tts.py
- | └── test_integration.py
- └── app.py
- └── requirements.txt
- └── .env.example
- └── .gitignore
- └── pytest.ini
- └── README.md
-
- requirements.txt에 다음 라이브러리를 포함해주세요:
- - streamlit
- - openai
- - gtts
- - pydub
- - pytest
- - pytest-cov
- - python-dotenv
- - requests
-

- .env.example 파일에 필요한 환경 변수 템플릿을 작성해주세요.
- pytest.ini 파일도 기본 설정으로 생성해주세요.

프롬프트 0-2: README 작성

- 프로젝트의 README.md 파일을 작성해주세요.
- 다음 내용을 포함해야 합니다:
 - - 프로젝트 개요
 - - 주요 기능
 - - 설치 방법
 - - 환경 설정 (.env 파일 설정)
 - - 실행 방법
 - - 테스트 실행 방법
 - - 프로젝트 구조 설명
 - - 기술 스택
 - - 라이선스

Phase 1: STT 기능 (TDD)

프롬프트 1-1: STT 테스트 케이스 작성

- TDD 방식으로 Whisper STT 기능을 구현하겠습니다.
- 먼저 tests/test_stt.py에 다음 테스트 케이스를 작성해주세요:
 - 1. test_whisper_service_initialization
 - - WhisperService 클래스가 올바르게 초기화되는지 테스트
 - - API 키가 없을 때 에러 발생 테스트
 - 2. test_audio_file_validation
 - - 지원되는 오디오 포맷(mp3, wav, m4a) 검증

- - 지원되지 않는 포맷 에러 처리
- - 파일 크기 제한 검증
-
- 3. test_transcribe_audio_success
- - 정상적인 오디오 파일을 텍스트로 변환
- - Mock을 사용하여 OpenAI API 호출 시뮬레이션
-
- 4. test_transcribe_audio_failure
- - API 호출 실패 시 에러 처리
- - 네트워크 에러 처리
-
- 5. test_language_detection
- - 한국어 음성 인식 테스트
- - 영어 음성 인식 테스트
-
- pytest와 unittest.mock을 사용하여 작성해주세요.

프롬프트 1-2: STT 서비스 구현

- 이제 tests/test_stt.py의 테스트를 통과하도록
- src/stt/whisper_service.py를 구현해주세요.
-
- WhisperService 클래스는 다음 메서드를 포함해야 합니다:
 - - __init__(self, api_key: str)
 - - validate_audio_file(self, file_path: str) -> bool
 - - transcribe(self, audio_file: str, language: str = "ko") -> str
 - - _handle_api_error(self, error: Exception) -> str
-
- 타입 힌팅을 사용하고, docstring을 추가해주세요.
- 에러 처리를 철저히 해주세요.

프롬프트 1-3: STT 테스트 실행 및 커버리지 확인

- 작성한 STT 테스트를 실행하고 커버리지를 확인해주세요.
 - 다음 명령어를 실행할 수 있도록 가이드해주세요:
 -
 - `pytest tests/test_stt.py -v`
 - `pytest tests/test_stt.py --cov=src.stt --cov-report=html`
 -
 - 커버리지가 80% 미만이면 추가 테스트 케이스를 제안해주세요.
-

Phase 2: GPT 연동 (TDD)

프롬프트 2-1: GPT 테스트 케이스 작성

- TDD 방식으로 GPT 서비스를 구현하겠습니다.
- `tests/test_llm.py`에 다음 테스트 케이스를 작성해주세요:
-
- 1. `test_gpt_service_initialization`
- - GPTService 클래스 초기화 테스트
- - API 키 검증
- - 모델 선택 검증 (`gpt-4`, `gpt-3.5-turbo`)
-
- 2. `test_generate_response_success`
- - 정상적인 질문에 대한 답변 생성
- - Mock을 사용한 API 호출 테스트
-
- 3. `test_generate_response_with_context`
- - 대화 맥락을 포함한 답변 생성
- - 멀티턴 대화 테스트
-
- 4. `test_generate_response_failure`

- - API 호출 실패 처리
- - Rate limit 에러 처리
- - Timeout 에러 처리
-
- 5. test_conversation_history_management
- - 대화 히스토리 추가
- - 대화 히스토리 조회
- - 대화 히스토리 초기화
-
- 6. test_token_limit_handling
- - 긴 대화 히스토리 처리
- - 토큰 제한 초과 시 처리
-
- pytest를 사용하여 작성해주세요.

프롬프트 2-2: GPT 서비스 구현

- tests/test_llm.py의 테스트를 통과하도록
- src/llm/gpt_service.py를 구현해주세요.
-
- GPTService 클래스는 다음 메서드를 포함해야 합니다:
- - __init__(self, api_key: str, model: str = "gpt-3.5-turbo")
- - generate_response(self, user_message: str, conversation_history: list = None) -> str
- - add_to_history(self, role: str, content: str) -> None
- - get_history(self) -> list
- - clear_history(self) -> None
- - _build_messages(self, user_message: str) -> list
- - _handle_token_limit(self) -> None
-
- 타입 힌팅, docstring, 에러 처리를 포함해주세요.

프롬프트 2-3: GPT 테스트 실행

- GPT 서비스 테스트를 실행하고 결과를 확인해주세요:
 -
 - `pytest tests/test_llm.py -v --cov=src.llm`
 -
 - 모든 테스트가 통과하는지 확인하고,
 - 실패하는 테스트가 있다면 수정 방안을 제시해주세요.
-

Phase 3: TTS 기능 (TDD)

프롬프트 3-1: TTS 테스트 케이스 작성

- TDD 방식으로 gTTS 서비스를 구현하겠습니다.
- `tests/test_tts.py`에 다음 테스트 케이스를 작성해주세요:
-
- 1. `test_tts_service_initialization`
 - - TTSService 클래스 초기화
- - 언어 설정 검증
-
- 2. `test_text_to_speech_success`
 - - 텍스트를 음성 파일로 변환
 - - 생성된 파일 존재 확인
 - - 파일 형식 검증 (mp3)
-
- 3. `test_text_to_speech_korean`
 - - 한국어 텍스트 음성 변환
 - - 발음 정확도 검증
-
- 4. `test_text_to_speech_failure`
 - - 빈 텍스트 처리

- - 너무 긴 텍스트 처리
- - 파일 생성 실패 처리
-
- 5. test_audio_file_cleanup
- - 임시 파일 정리 기능
- - 오래된 파일 자동 삭제
-
- 6. test_language_support
- - 다국어 지원 테스트 (ko, en, ja)
-
- pytest를 사용하여 작성해주세요.

프롬프트 3-2: TTS 서비스 구현

- tests/test_tts.py의 테스트를 통과하도록
- src/tts/gtts_service.py를 구현해주세요.
-
- TTSService 클래스는 다음 메서드를 포함해야 합니다:
 - - __init__(self, language: str = "ko")
 - - text_to_speech(self, text: str, output_file: str = None) -> str
 - - validate_text(self, text: str) -> bool
 - - cleanup_temp_files(self, directory: str = "./temp") -> None
 - - set_language(self, language: str) -> None
-
- 타입 힌팅, docstring, 에러 처리를 포함해주세요.
- 임시 파일은 ./temp 디렉토리에 저장되도록 해주세요.

프롬프트 3-3: TTS 테스트 실행

- TTS 서비스 테스트를 실행해주세요:
-
- `pytest tests/test_tts.py -v --cov=src.tts`
-

- 테스트 통과 여부와 커버리지를 확인해주세요.

Phase 4: Streamlit UI 구현

프롬프트 4-1: 기본 UI 구조 생성

- app.py에 Streamlit 기반 UI를 구현해주세요.
- 다음 구조를 따라주세요:
 - - 1. 페이지 설정
 - - 타이틀: "🗣️ ChatGPT - 음성 비서"
 - - 레이아웃: wide
 - - 사이드바 설정
 - 2. 사이드바 구성
 - - API Key 입력 (password 타입)
 - - 모델 선택 (gpt-4, gpt-3.5-turbo)
 - - TTS 언어 선택 (한국어, 영어)
 - - 대화 초기화 버튼
 - 3. 메인 레이아웃 (2컬럼)
 - - 왼쪽: 질문 입력 영역
 - - 오른쪽: 답변 출력 영역
 - 4. 세션 상태 초기화
 - - conversation_history
 - - audio_files
 - st.session_state를 활용해주세요.

프롬프트 4-2: 음성 입력 UI 구현

- app.py의 왼쪽 컬럼에 음성 입력 기능을 추가해주세요:
-
- 1. 오디오 파일 업로더
 - - 지원 형식: mp3, wav, m4a
 - - 파일 크기 제한: 10MB
-
- 2. 업로드된 오디오 재생
 - - st.audio()를 사용
-
- 3. "음성을 텍스트로 변환" 버튼
 - - 클릭 시 WhisperService 호출
 - - 로딩 스피너 표시
 - - 변환된 텍스트 표시
-
- 4. 수동 텍스트 입력 옵션
 - - st.text_area()로 직접 질문 입력 가능
-
- src/stt/whisper_service.py를 import하여 사용해주세요.

프롬프트 4-3: GPT 답변 생성 UI 구현

- app.py의 오른쪽 컬럼에 답변 생성 기능을 추가해주세요:
-
- 1. "답변 생성" 버튼
 - - 질문이 입력되었을 때만 활성화
 - - 클릭 시 GPTService 호출
 - - 로딩 스피너 표시

-
- 2. 답변 텍스트 표시
 - - `st.markdown()`으로 포매팅
 - - 답변 영역 스타일링
-
- 3. "음성으로 듣기" 버튼
 - - 클릭 시 `TTSService` 호출
 - - 생성된 오디오 자동 재생
 - - 다운로드 버튼 제공
-
- 4. 에러 처리
 - - API 에러 시 `st.error()` 메시지
 - - 재시도 옵션 제공
-
- `src/llm/gpt_service.py`와 `src/tts/gtts_service.py`를 `import`하여 사용해주세요.

프롬프트 4-4: 채팅 히스토리 UI 구현

- `app.py`에 채팅 히스토리 기능을 추가해주세요:
-
- 1. 히스토리 표시 영역
 - - 메인 컬럼 하단에 배치
 - - `st.expander()`로 접고 펼치기 가능
-
- 2. 대화 기록 포맷
 - - 각 질문-답변 쌍을 카드 형태로 표시
 - - 타임스탬프 포함
 - - 음성 파일 재생 버튼

-
- 3. 대화 내보내기
- - JSON 형식으로 다운로드
- - 버튼: "대화 내역 다운로드"
-
- st.session_state의 conversation_history를 활용해주세요.

Phase 5: 통합 및 리팩토링

프롬프트 5-1: 통합 테스트 작성

- tests/test_integration.py에 통합 테스트를 작성해주세요:
-
- 1. test_end_to_end_conversation
- - STT → GPT → TTS 전체 플로우 테스트
- - Mock 데이터 사용
-
- 2. test_error_handling_integration
- - 각 단계에서 에러 발생 시 처리
- - 부분 실패 시나리오
-
- 3. test_session_state_management
- - 대화 히스토리 저장 및 조회
- - 세션 초기화
-
- 4. test_file_cleanup
- - 임시 파일 정리 확인
-
- pytest를 사용하여 작성해주세요.

프롬프트 5-2: 코드 리팩토링 - DRY 원칙

- 전체 코드를 검토하고 다음 사항을 리팩토링해주세요:
-
- 1. 중복 코드 제거
- - 예러 처리 로직 통합
- - 유틸리티 함수 분리
-
- 2. 공통 로직 추출
- - src/utils/audio_utils.py에 오디오 관련 유틸리티 함수 추가
- - src/utils/error_handler.py 생성
-
- 3. 매직 넘버/스트링 제거
- - 상수를 별도 파일로 관리
- - src/config.py 생성
-
- 4. 타입 힌팅 추가
- - 모든 함수에 타입 힌팅 적용
- - from typing import Optional, List, Dict 활용
-
- 리팩토링 후 모든 테스트가 통과하는지 확인해주세요.

프롬프트 5-3: 코드 품질 향상

- 코드 품질을 향상시켜주세요:
-
- 1. Docstring 추가
- - 모든 클래스와 메서드에 Google 스타일 docstring 추가
- - 파라미터, 반환값, 예외 설명 포함
-
- 2. 로깅 추가

- - logging 모듈 사용
- - INFO, WARNING, ERROR 레벨 로깅
- - src/utils/logger.py 생성
-
- 3. 설정 관리 개선
- - dataclass를 사용한 설정 클래스 생성
- - 환경 변수와 기본값 관리
-
- 4. 에러 메시지 개선
- - 사용자 친화적인 에러 메시지
- - 한국어 메시지 지원
-
- 구체적인 개선 사항을 적용해주세요.

프롬프트 5-4: 성능 최적화

- 애플리케이션 성능을 최적화해주세요:
-
- 1. 캐싱 적용
- - @st.cache_data 데코레이터 활용
- - API 호출 결과 캐싱
-
- 2. 비동기 처리
- - asyncio를 사용한 병렬 처리
- - STT와 TTS 동시 처리 가능 여부 검토
-
- 3. 파일 처리 최적화
- - 임시 파일 즉시 정리

- - 메모리 사용량 최소화
-
- 4. UI 응답성 개선
- - st.spinner() 활용
- - 프로그레스 바 추가
-
- 성능 개선 사항을 구현하고 테스트해주세요.

프롬프트 5-5: 최종 테스트 및 문서화

- 프로젝트를 최종 점검해주세요:
-
- 1. 전체 테스트 실행
- `pytest tests/ -v --cov=src --cov-report=html --cov-report=term`
-
- 2. 테스트 커버리지 확인
- - 목표: 80% 이상
- - 부족한 부분 추가 테스트 작성
-
- 3. README.md 업데이트
- - 최신 기능 반영
- - 스크린샷 추가
- - 사용 예시 추가
-
- 4. 문서 작성
- - docs/ 폴더 생성
- - API 문서 작성
- - 개발자 가이드 작성
-

- 5. 배포 준비
 - - requirements.txt 최종 확인
 - - .gitignore 점검
 - - 라이선스 파일 추가
 -
 - 최종 체크리스트를 제공해주세요.
-

🔗 TDD 워크플로우 요약

- 각 Phase에서 다음 사이클을 반복합니다:
 - 1. RED 단계: 테스트 작성 (실패하는 테스트)
 - ↓
 - 2. GREEN 단계: 최소한의 코드로 테스트 통과
 - ↓
 - 3. REFACTOR 단계: 코드 개선 및 리팩토링
 - ↓
 - 4. 반복
-

📖 각 단계별 체크리스트

Phase 1 체크리스트

- STT 테스트 케이스 작성 완료
- WhisperService 구현 완료
- 모든 테스트 통과 (pytest)
- 코드 커버리지 80% 이상
- 타입 힌팅 적용
- Docstring 작성

Phase 2 체크리스트

- GPT 테스트 케이스 작성 완료
- GPService 구현 완료
- 대화 히스토리 관리 기능
- 모든 테스트 통과
- 에러 처리 구현

Phase 3 체크리스트

- TTS 테스트 케이스 작성 완료
- TTService 구현 완료
- 임시 파일 관리 기능
- 모든 테스트 통과
- 다국어 지원

Phase 4 체크리스트

- Streamlit UI 기본 구조
- 음성 입력 UI
- 답변 생성 UI
- 채팅 히스토리 UI
- 사이드바 설정
- UI/UX 테스트

Phase 5 체크리스트

- 통합 테스트 작성
- 코드 리팩토링 완료
- 로깅 시스템 구현
- 성능 최적화
- 문서화 완료

- 배포 준비 완료

💡 Cursor AI 활용 팁

1. 컨텍스트 제공

- 프롬프트 시작 시 항상 다음을 언급:
- 우리는 현재 ChatGPT 프로젝트의 [Phase X]를 진행 중입니다.
- TDD 방식을 따르고 있으며, [이전 단계에서 완료한 내용]을 바탕으로
- [현재 작업]을 진행하려고 합니다.

2. 명확한 요구사항

- 구체적인 클래스명, 메서드명 명시
- 예상되는 입력/출력 예시 제공
- 에러 케이스 명시

3. 점진적 개발

- 한 번에 하나의 기능만 구현
- 각 기능마다 테스트 먼저 작성
- 작은 단위로 커밋

4. 코드 리뷰 요청

- 방금 작성한 코드를 리뷰해주세요.
- 다음 관점에서 검토해주세요:
 - - 코드 품질
 - - 테스트 커버리지
 - - 에러 처리
 - - 성능
 - - 보안

5. 리팩토링 요청

- [파일명]을 리팩토링해주세요.
 - 다음 원칙을 따라주세요:
 - - SOLID 원칙
 - - DRY 원칙
 - - 가독성 향상
 - - 성능 최적화
-

시작하기

1. Phase 0부터 순차적으로 진행
 2. 각 프롬프트를 Cursor AI에 복사하여 실행
 3. 생성된 코드를 검토하고 테스트
 4. 문제가 있으면 수정 요청
 5. 다음 단계로 진행
-

참고 자료

- [Pytest 문서](#)
 - [Streamlit 문서](#)
 - [OpenAI API 문서](#)
 - [TDD 모범 사례](#)
-

- **Good Luck with Your Development!** 🍀
- 각 단계를 차근차근 진행하시면 견고한 음성 비서 애플리케이션을 완성할 수 있습니다!

Awesome—let's drive **refactoring with TDD** in Cursor.AI using your ChatGPT PRD.

아래는 그대로 복사/붙여넣기 해도 되는 **단계별 프롬프트 체인**입니다.

각 단계는 **RED → GREEN → REFACTOR** 사이클을 유지하도록 설계했어요.

🔗 Phase 0. 컨텍스트 주입 & 작업 규칙

- **Prompt 0-1 — 컨텍스트 로딩(필수)**

- 1. 당신은 선임 Python 엔지니어이자 테스트 리더입니다.
 2. 다음 PRD를 읽고 ChatGPT(Whisper + GPT + gTTS를 사용하는 Streamlit 음성 비서)의 권위 있는 맥락으로 기억하십시오.
 3. 핵심 목표: TDD 우선, 깔끔한 아키텍처, 높은 응집도/낮은 결합도, 90% 이상 커버리지, 빠른 피드백
 4. [여기에 ChatGPT PRD 마크다운 붙여넣기]
 5. 준비가 되면 알려주세요. 아직 코드를 생성하지 마세요.

- **Prompt 0-2 — 작업 규칙 확정**

- 1. 이후 모든 단계에 다음 규칙을 적용하세요.
 2. - Python 3.10 이상, Streamlit, pytest, pytest-cov, ruff/black/isort를 사용하세요.
 3. - 구조: app/{ui.py, stt.py, llm.py, tts.py, audio.py, config.py, utils.py}, tests/, .github/workflows/ci.yml
 4. - 외부 호출(OpenAI, gTTS)은 썬 래퍼(thin wrapper)로 추상화하고 테스트에서 모의해야 합니다.
 5. - 모든 리팩토링은 테스트를 녹색 상태로 유지해야 합니다. 교체 없이 테스트를 삭제하지 마세요.
 6. 계획을 확인하고 가정 사항을 나열하세요.

1. .

2.

Phase 1. 스캐폴딩 & 테스트 환경 (GREEN 목표 아님: 테스트만 생성)

- Prompt 1-1 — 프로젝트 뼈대 + 의존성

-

3. 생성:

4. - pyproject.toml (black, isort, ruff 구성됨)

5. - requirements.txt (streamlit, openai, gtts, python-dotenv, ffmpeg-python, pytest, pytest-cov)

6. - src 레이아웃: app/ (빈 모듈 스텝), tests/ (빈 파일)

7. - addopts = "-q --cov=app --cov-report=term-missing"을 사용한 pytest.ini

8. 트리 뷰와 파일 내용을 반환합니다.

3.

- Prompt 1-2 — 테스트 전용 인터페이스 추상화

-

4. 최소한의 시그니처로 얇은 인터페이스 래퍼를 만듭니다.

5. - app/stt.py: def transcribe(audio_path: str) -> str

6. - app/llm.py: def answer(prompt: str, *, system_prompt: str | None = None) -> str

7. - app/tts.py: def synthesize(text: str, *, lang: str = "ko") -> str # mp3 경로 반환

8. docstring과 TODO "impl later"를 추가합니다.

9. 지금은 로직을 구현하지 않습니다. NotImplementedError를 발생시킵니다.

4.

Phase 2. RED — 핵심 기능의 실패하는 테스트 작성

- 프롬프트 2-1 — STT/LLM/TTS 단위 RED

5. 실패하는 단위 테스트 작성:

6. - tests/test_stt.py: 주어진 "tests/data/hello.wav"를 transcribe()하여 "안녕하세요" (모의 Whisper)

7. - tests/test_llm.py: answer("날씨 어때?")는 비어 있지 않은 문자열을 반환하고 ≤ 512자

(모의 OpenAI)를 반환합니다.

8. - tests/test_tts.py: synthesize("테스트")는 기존 .mp3 경로를 반환하고 컨텍스트 종료 시 삭제합니다.

9. pytest monkeypatch/fixtures를 사용합니다. 실제 API를 호출하지 않습니다. 네트워크가 없습니다.

10. tmp_path를 통해 작은 가짜 wav 바이트를 포함합니다(바이너리 에셋 필요 없음).

- 프롬프트 2-2 — 세션/흐름 RED

11. 대화 흐름에 대한 테스트 추가(아직 Streamlit UI 없음):

12. - tests/test_flow.py

13. - 주어진 user_speech.wav -> transcribe() -> answer() -> synthesize()

14. - dict 유사 "session"에 저장된 중간 텍스트와 mp3 경로가 존재하며 play() 후 삭제됨

15. 간단한 메모리 내 세션(dict) 픽처를 설계합니다.

- 프롬프트 2-3 — 오류/예외 RED

16. 다음을 포함하는 실패 테스트를 추가합니다.

17. - 오디오가 비어 있거나 너무 짧으면 ValueError("음성이 감지되지 않음") 발생

18. - llm 시간 초과 -> "LLM 호출 실패" 메시지와 함께 RuntimeError 발생

19. - tts 텍스트가 비어 있으면 None을 반환하고 경고를 기록합니다.

20. 경고/오류에 대한 caplog 기반 어설션을 만듭니다.

● Phase 3. GREEN — 최소 구현으로 테스트 통과

- 프롬프트 3-1 — STT 최소 구현

5. app/stt.transcribe()를 구현합니다.

6. - audio_path를 받고, 100바이트보다 큰 파일이 있는지 확인하고, 그렇지 않으면 ValueError를 발생시킵니다.

7. - OPENAI_API_KEY가 없는 경우 오프라인 모의 경로를 시뮬레이션합니다(env 플래그로 보호됨).

8. - 현재로서는 몽키패치 시 결정적 플레이스홀더를 반환하고, 그렇지 않으면 오류를 발생시킵니다.

9. 적절한 몽키패치 지점을 통해 테스트 2-1을 통과시킵니다.

- 프롬프트 3-2 — LLM 최소 구현

10. `app/llm.answer()`를 구현합니다.

11. - `prompt/system_prompt`를 받습니다.

12. - `env/config`에서 `OPENAI_API_KEY`와 모델을 읽습니다.

13. - `_chat(api_key, model, prompt, system_prompt)` 함수로 래핑된 실제 호출

14. - 하지만 `_chat`의 몽키패치를 허용하여 테스트 통과를 보장합니다.

15. 512자 이하의 문자열을 반환합니다. 필요한 경우 잘라냅니다.

- 프롬프트 3-3 — TTS 최소 구현 + 임시파일 수명

16. `app/tts.synthesize()`를 구현합니다.

17. - 텍스트가 거짓이면 -> 경고를 기록하고 `None`을 반환합니다.

18. - 그렇지 않으면 임시 디렉터리에 임시 mp3를 생성합니다.

19. - 컨텍스트 관리자 헬퍼를 제공합니다. `speak_file(path)` -> 경로를 생성한 후 종료 시 삭제합니다.

20. `safe_delete(path)`를 사용하여 `app/utils.py`를 작성합니다.

21. 파일 존재 및 삭제 테스트를 통과시킵니다.

- 프롬프트 3-4 — 흐름 유틸

22. `app/utils.py` 헬퍼를 생성합니다.

23. - `now_ts()` -> `int`

24. - `ensure_dir(p)` -> `Path`

25. - `tts.synthesize` + 삭제를 사용하는 `with_temp_audio(text)` 컨텍스트 관리자

26. 필요한 경우 이러한 헬퍼를 가져오도록 테스트를 업데이트합니다.

Phase 4. REFACTOR — 구조 개선(테스트 유지)

프롬프트 4-1 — 중복 제거 및 의존성 역전

5. 리팩토링:

6. - app/config.py에 dataclass AppConfig(api_key, model, tts_lang)를 도입합니다.

7. - 가능하면 stt/llm/tts 함수에 구성 객체를 전달합니다.

8. - 작은 함수의 부작용을 추출합니다. 비즈니스 규칙의 경우 순수 함수로 처리합니다.

9. 모든 테스트를 안전하게 유지합니다. 차이점과 근거를 제공합니다.

• 프롬프트 4-2 — 메시지 오류 해결

10. 형식화된 예외를 사용하여 app/errors.py를 생성합니다.

11. - NoSpeechError, LLMError, AudioloError

12. 일반적인 예외 발생을 다음으로 대체합니다. 문자열이 아닌 클래스를 예상하도록 테스트를 업데이트합니다.

13. UX를 위해 str(exception)에 이전 메시지를 유지합니다.

• 프롬프트 4-3 — 예외 처리

14. app.utils.get_logger(name)에 구조화된 로깅을 추가합니다.

15. print를 logger로 바꾸세요. Caplog 테스트가 여전히 통과하는지 확인하세요.

🔧 Phase 5. 커버리지 확대 & 회귀 방지

• 프롬프트 5-1 — 커버리지 갭 분석

5. "pytest --cov=app --cov-report=term-missing"을 실행합니다.

6. 커버리지되지 않은 줄/분기를 나열합니다. 90% 이상의 커버리지를 달성하기 위한 추가 테스트를 제안합니다.

7. 단언을 약화시키지 않고 지금 해당 테스트를 작성합니다.

• 프롬프트 5-2 — 경계/에지 RED→GREEN

8. 다음에 대한 테스트를 추가합니다.

9. - 매우 긴 LLM 출력 -> 512자로 잘림

10. - TTS에 대한 UTF8이 아닌 텍스트 입력 -> 경고와 함께 정규화되거나 안전한 실패

11. - STT에 대한 존재하지 않는 파일 경로 -> AudioloError

12. 통과하기 위한 최소한의 변경 사항을 구현합니다.

🔗 Phase 6. Streamlit UI 연동 (가벼운 통합)

- **Prompt 6-1 — UI 계약 테스트(RED)**

5. 다음을 어설션하는 tests/test_ui_contract.py를 생성합니다.

6. - ui.render()는 다음에 대한 자리 표시자가 있는 레이아웃 객체를 반환합니다.

7. - 사이드바: API 키 입력, 모델 선택, 재설정 버튼

8. - 메인 영역: 채팅 컨테이너(왼쪽: 사용자, 오른쪽: 봇)

9. Streamlit을 실행하지 않고 함수 계약/반환 마커만 테스트합니다.

- **Prompt 6-2 — UI 최소 구현(GREEN)**

10. app/ui.py를 구현합니다.

11. - render(config: AppConfig) -> 생성된 구성 요소(id, 유형)를 설명하는 Dict[str, Any]

12. - 내부에 네트워크 호출 없음. Streamlit 런타임에서 사용하는 메타데이터를 반환하는 순수 레이아웃 팩토리

13. 계약 테스트를 통과시킵니다.

- **Prompt 6-3 — 이벤트 바인딩(리팩터)**

- 14. UI를 분리하여 리팩토링합니다.

- 15. - 레이아웃 팩토리(순수)

- 16. - 이벤트 핸들러: on_record(), on_submit(), on_reset()

- 17. 핸들러는 주입된 함수를 통해 stt/llm/tts를 호출하여 모의 실행이 가능하도록 합니다.

- 18. 테스트를 녹색으로 유지합니다. DI 예시를 제공합니다.

5. .

🔗 Phase 7. 품질 자동화 & CI

- 프롬프트 7-1 — 린트/포맷 도입

6. pyproject.toml과 Makefile에 ruff/black/isort 설정을 추가합니다.

7. make test, make lint, make fmt, make cov

8. 업데이트된 파일을 반환합니다.

- 프롬프트 7-2 — GitHub Actions

9. .github/workflows/ci.yml을 생성합니다.

10. - matrix: ubuntu-latest, Python 3.10

11. - steps: checkout, setup-python, pip cache, install, black --check, ruff, pytest --cov

12. - main에서: 커버리지 아티팩트 업로드

13. yaml을 반환합니다.

Phase 8. 성능/자원 & 가드(선택)

- 프롬프트 8-1 — 성능 가드

6. pytest 마커를 사용하여 성능 테스트를 추가합니다.

7. - test_llm_latency_budget: answer() 모의 경로가 0.2초 미만인지 확인합니다.

8. - test_tts_temp_cleanup: 50회 반복 후 임시 파일이 0개 이상 생성되지 않습니다.

9. 실패할 경우 테스트 및 최소한의 최적화를 제공합니다.

6. .

Phase 9. 보안/설정 하드닝

- Prompt 9-1 — 설정 보안

7. 구성 로딩 구현(python-dotenv):

8. - OPENAI_API_KEY, MODEL, TTS_LANG을 포함하는 .env.sample

9. - app/config.py: load_env(), 로깅 시 api_key 삭제

10. api_key가 로그에 나타나지 않도록 하는 테스트를 추가합니다.

📖 Phase 10. 문서화 & 변경 로그

- Prompt 10-1 — README/개발자 가이드

7. README.md 생성:

8. - 개요, 설정, Streamlit 실행 방법, 테스트, Linting, CI, 아키텍처 다이어그램(ASCII), 제한 사항, 로드맵

9. 간결하고 복사하여 붙여넣기 가능하도록 작성하세요.

- Prompt 10-2 — 변경 로그

7. Keep a Changelog 형식을 사용하여 CHANGELOG.md를 생성합니다.

8. 다음 항목을 추가합니다.

9. - 0.1.0: 초기 TDD 스캐폴딩

10. - 0.2.0: DI + 타이핑 오류 + 커버리지 90%

11. - 0.3.0: UI 계약 + CI 파이프라인

🔄 반복 사이클용 짧은 프롬프트(즐거찾기)

- 실패 테스트 추가

7. [동작 설명]을 캡처하는 실패하는 pytest를 작성합니다.

8. 순수 함수를 선호하고 외부 I/O를 모의합니다. 단언을 엄격하게 유지합니다.

- 최소 구현

9. 새로운 실패 테스트만 통과하는 최소한의 코드를 구현합니다.

10. 부작용을 피하고 테스트를 녹색으로 유지합니다.

- 리팩토링

11. 중복을 제거하고 SRP/OCP를 적용하기 위해 [모듈]을 리팩토링합니다.

12. 동작을 변경하지 않습니다. 모든 테스트는 녹색으로 유지해야 합니다. 차이점과 근거를 보여줍니다.

- 커버리지 향상

13. 커버리지 갭을 분석하고 단언을 완화하지 않고 테스트되지 않은 브랜치를 포괄하는 집중적인 테스트를 생성합니다.

14. 90% 이상을 목표로 합니다.

☒ 기대 산출물 체크리스트

- 90%+ 커버리지의 **pytest** 스위트
- DI/래퍼 기반 모킹 친화 구조
- 표준화된 에러 클래스 + 구조화 로깅
- **Streamlit** UI 계약 테스트 통과
- **GitHub Actions** CI 파이프라인